Laboratory work 7

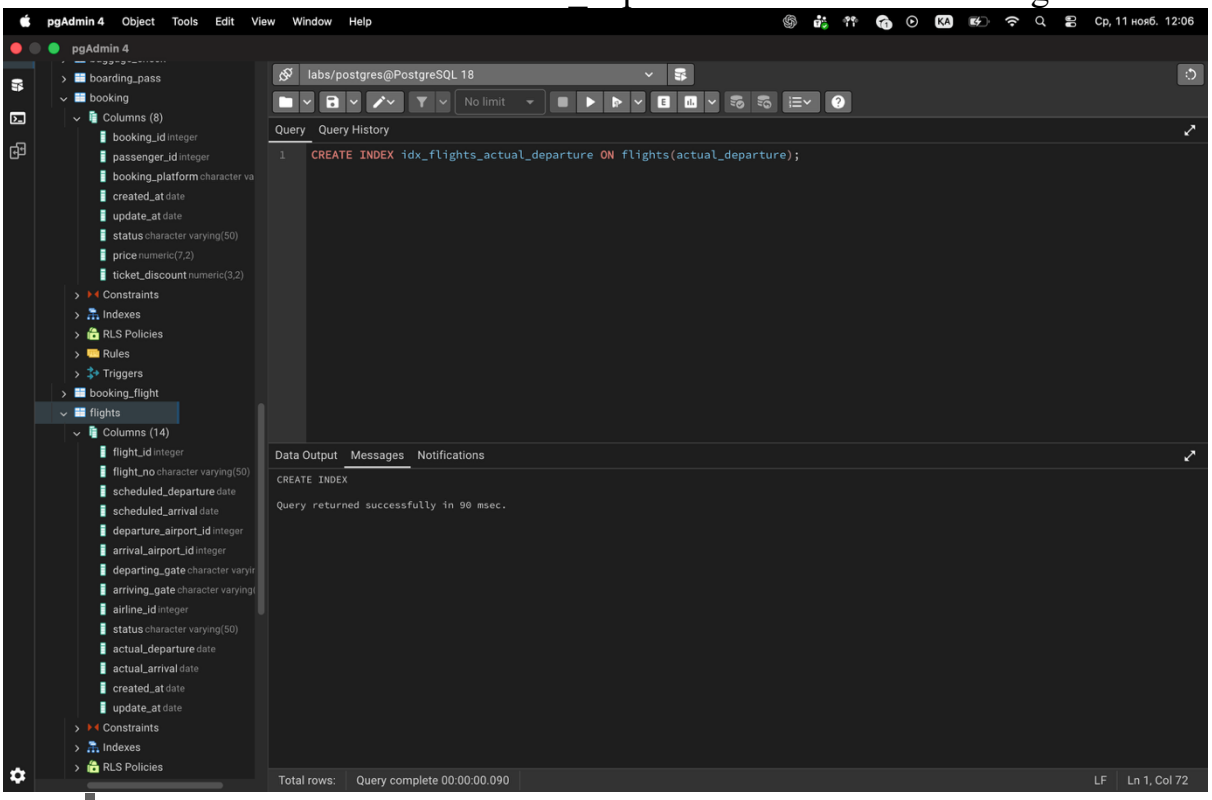
We continue to work with the database from the previous laboratory works.

Take a full-page screenshot that covers the code and results of each task.

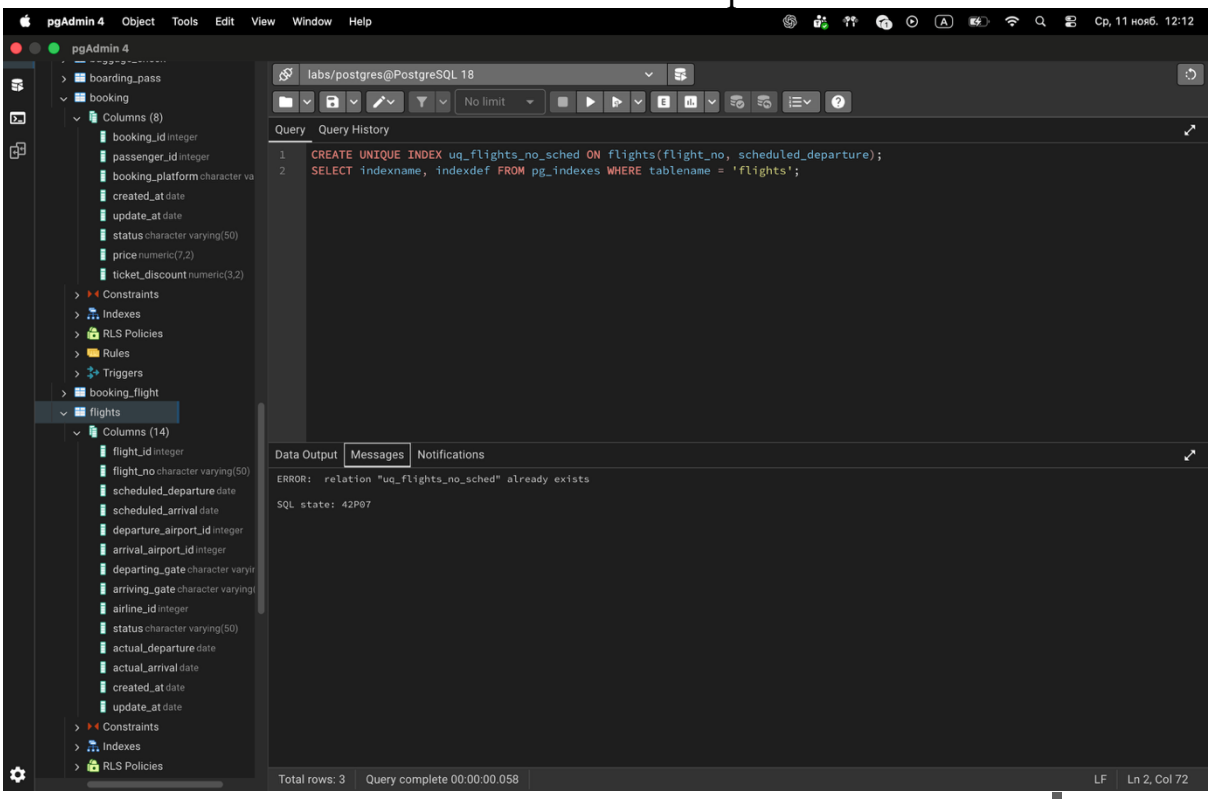
Tasks:

INDEX

1. Create an index on the actual_departure column in the flights table.

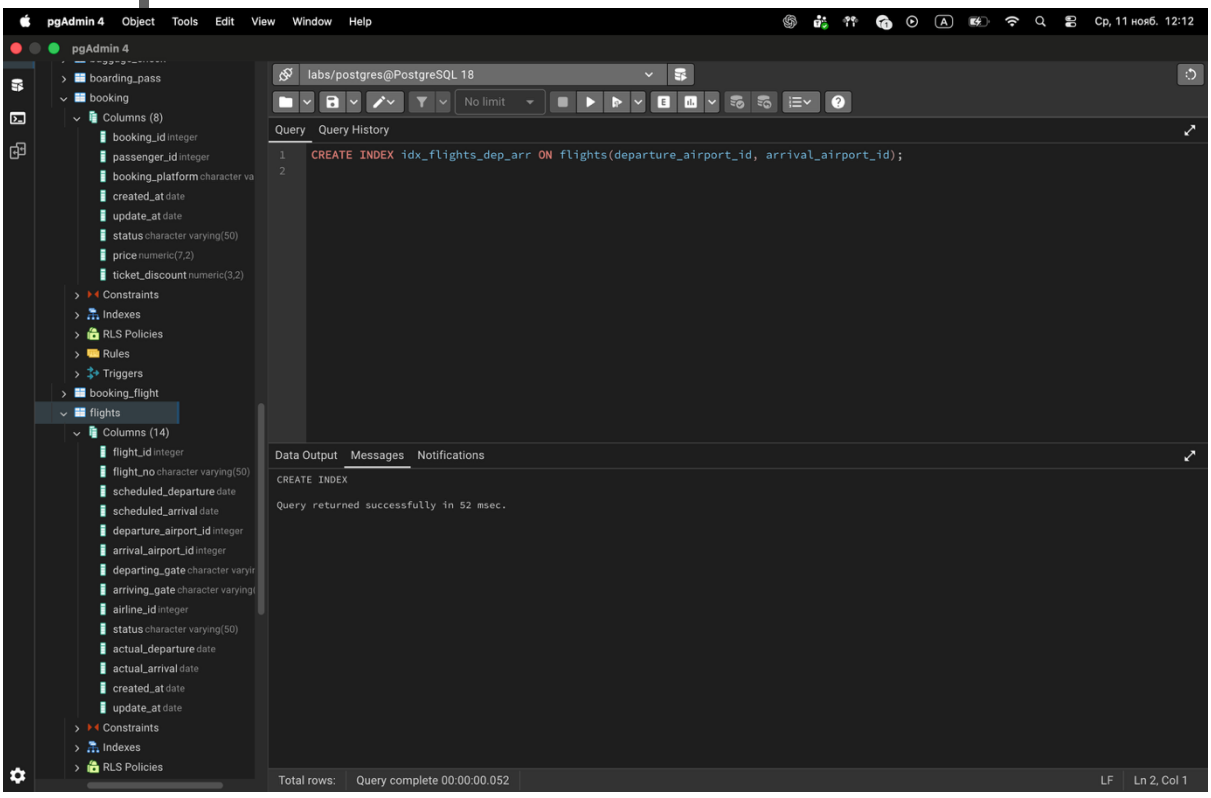


2. Create a unique index

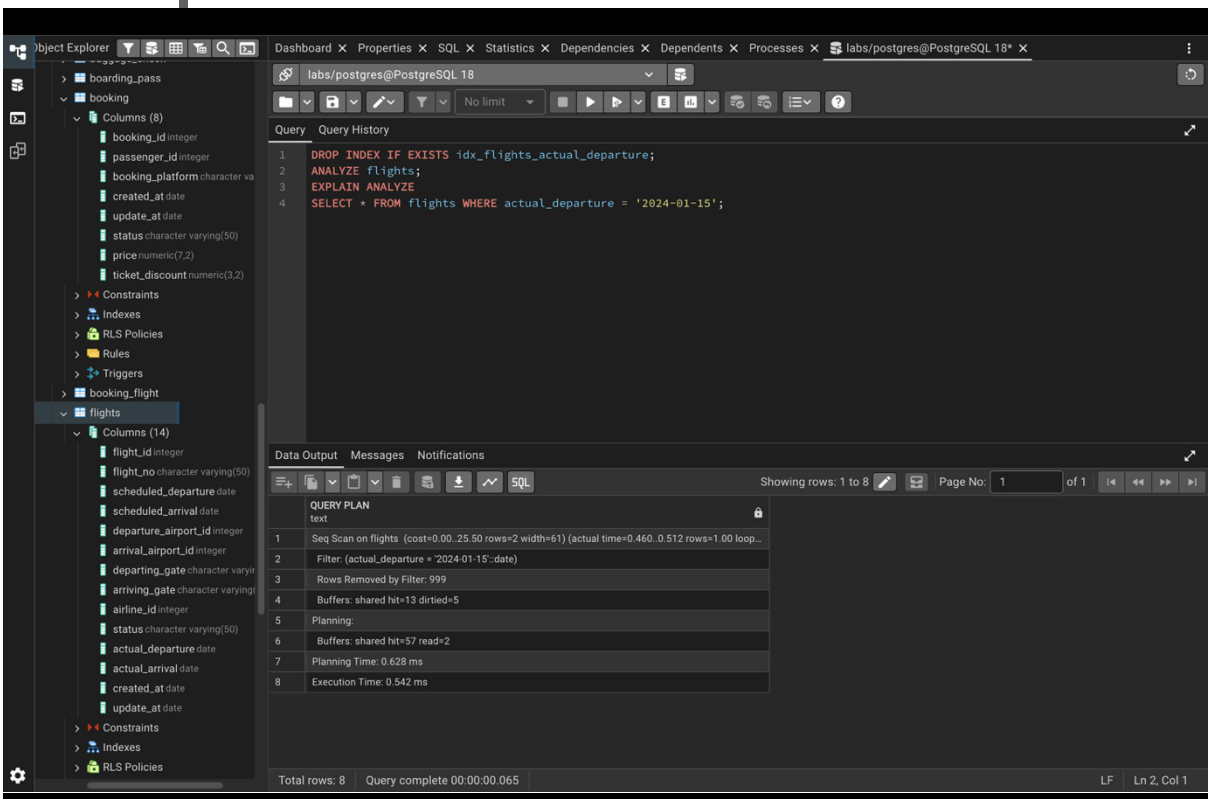


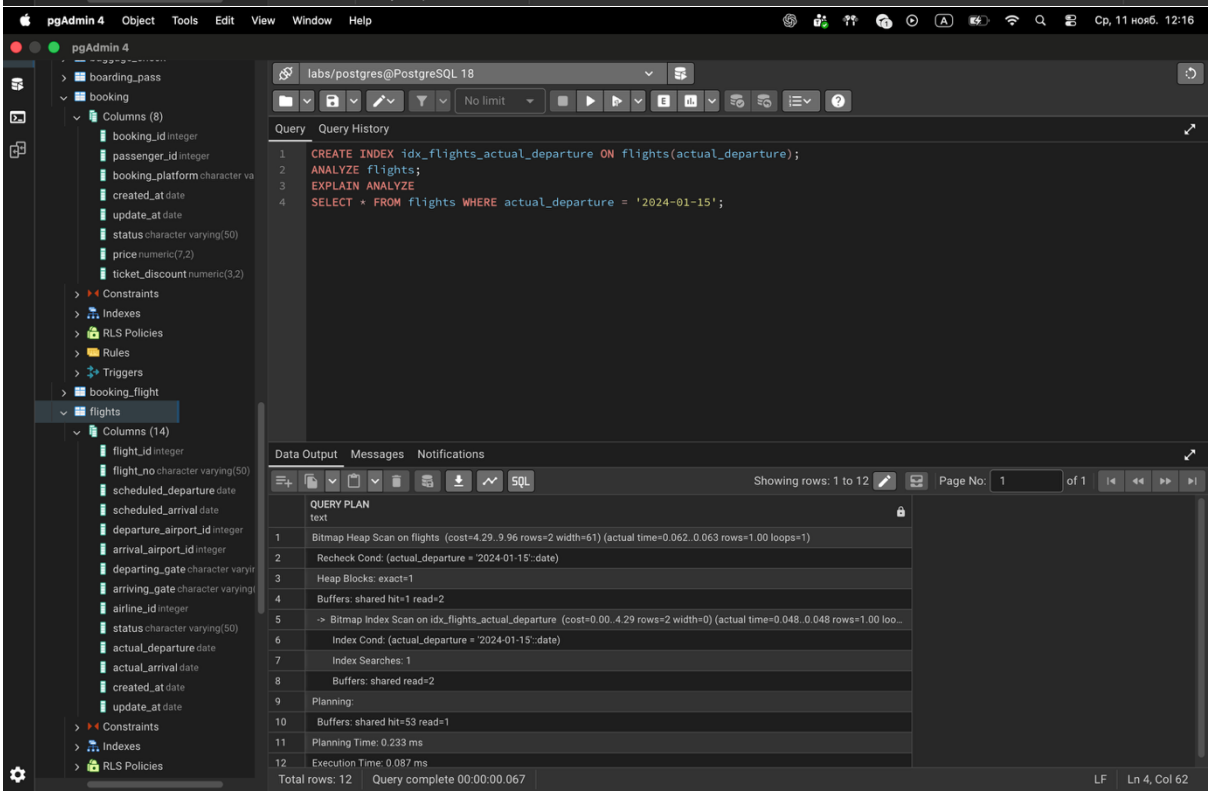
to ensure flight_no and scheduled_departure combinations are unique.

3. Create a composite index on the departure_airport_id and arrival_airport_id columns.

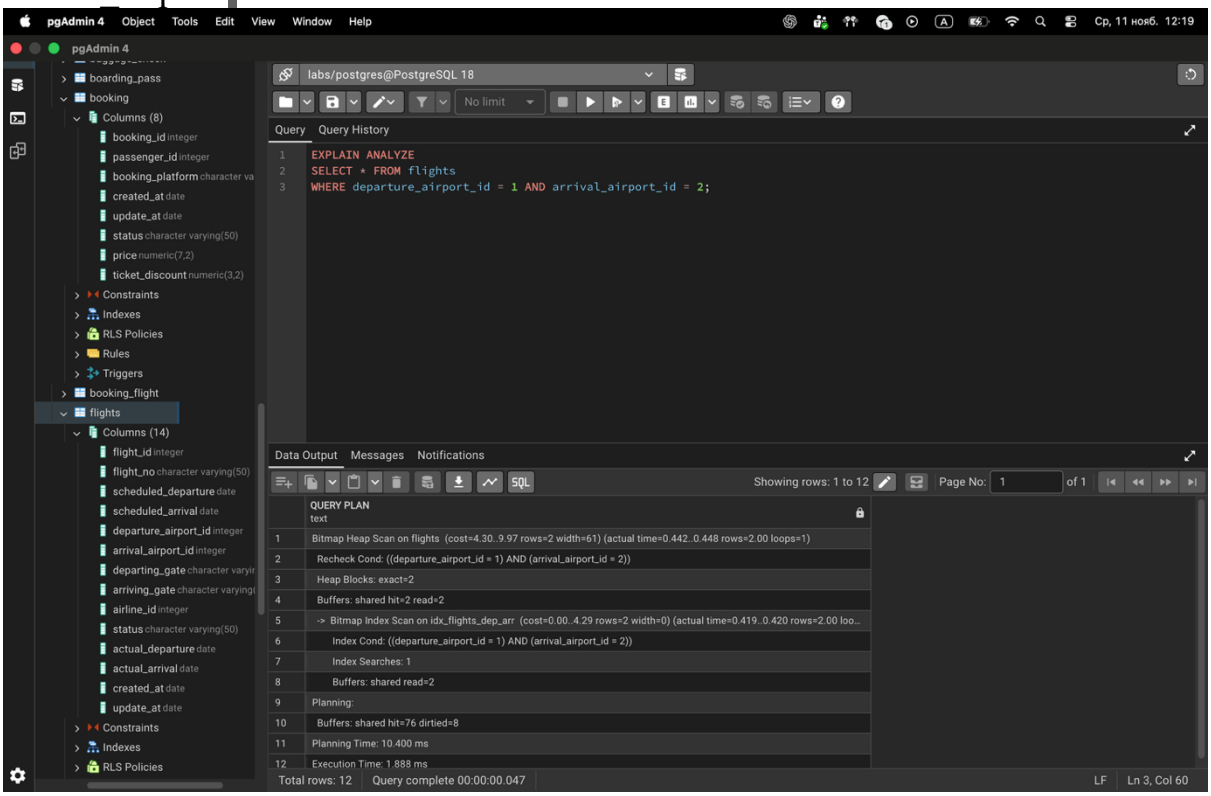


4. Evaluate the difference in query performance with and without indexes. Measure performance differences.

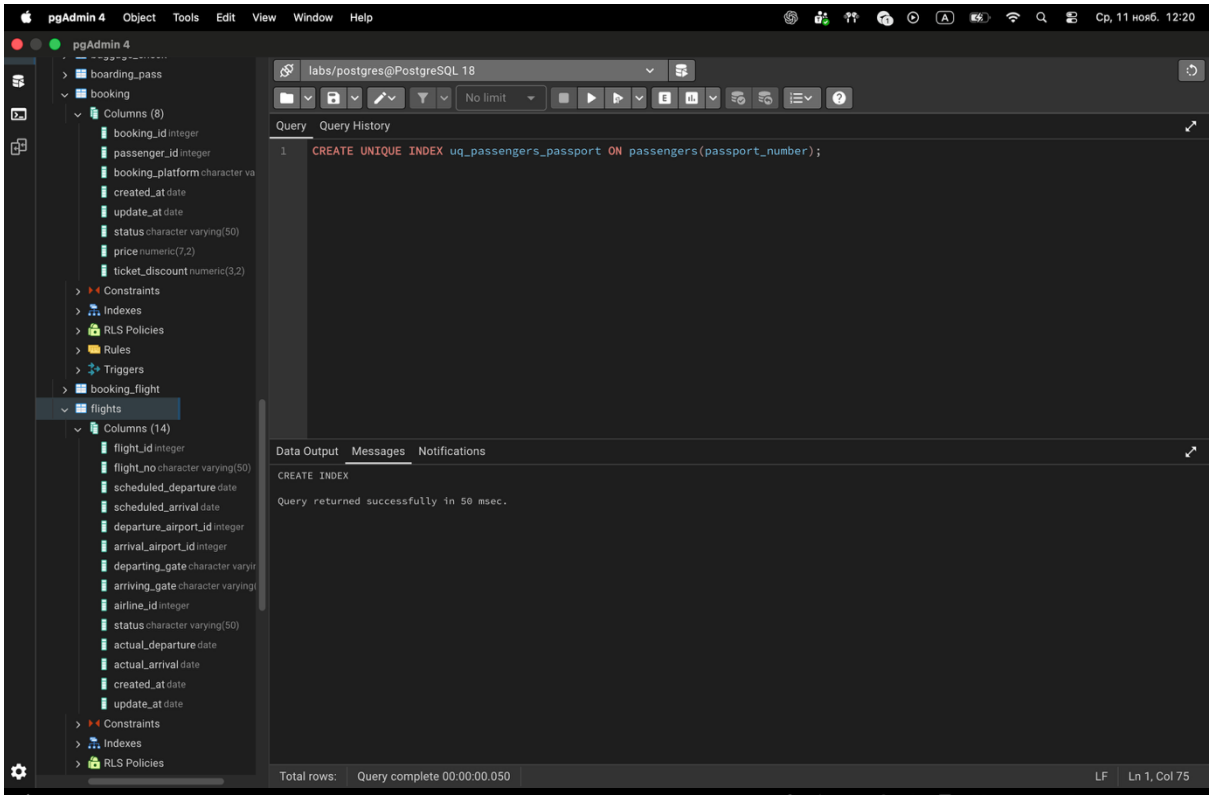


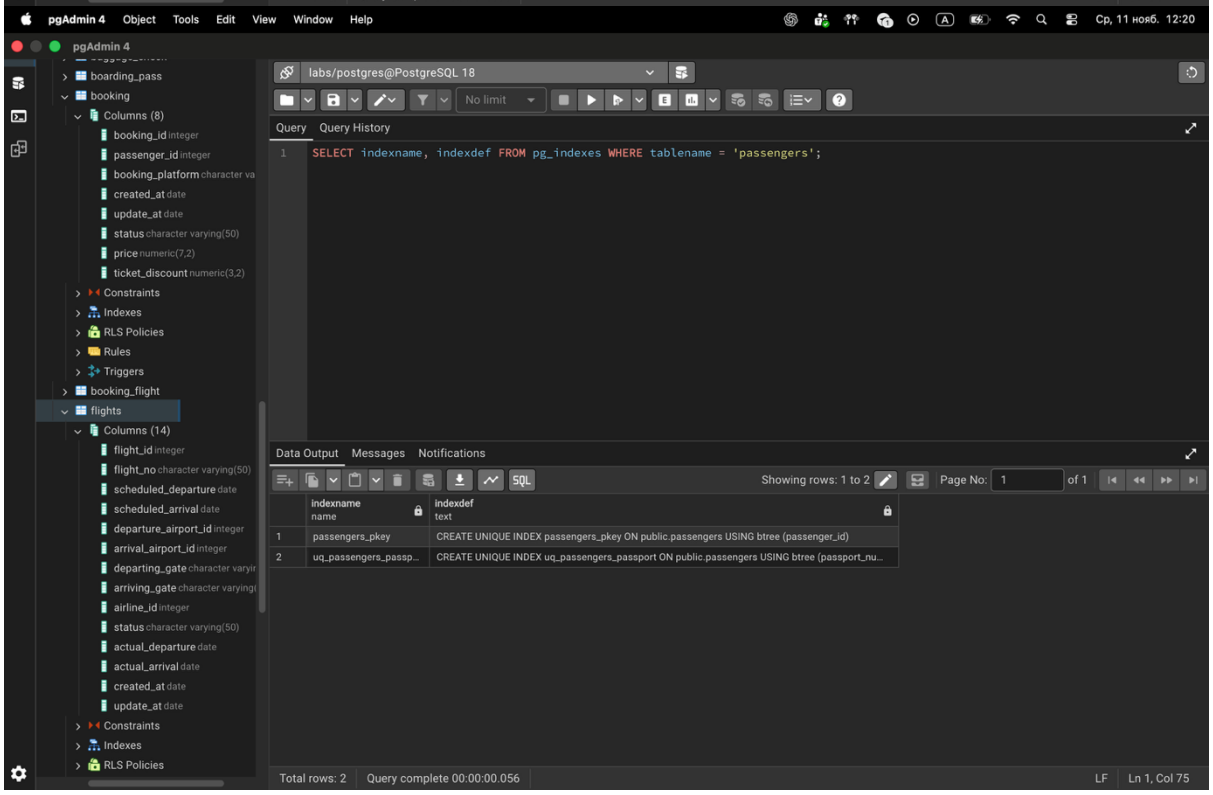


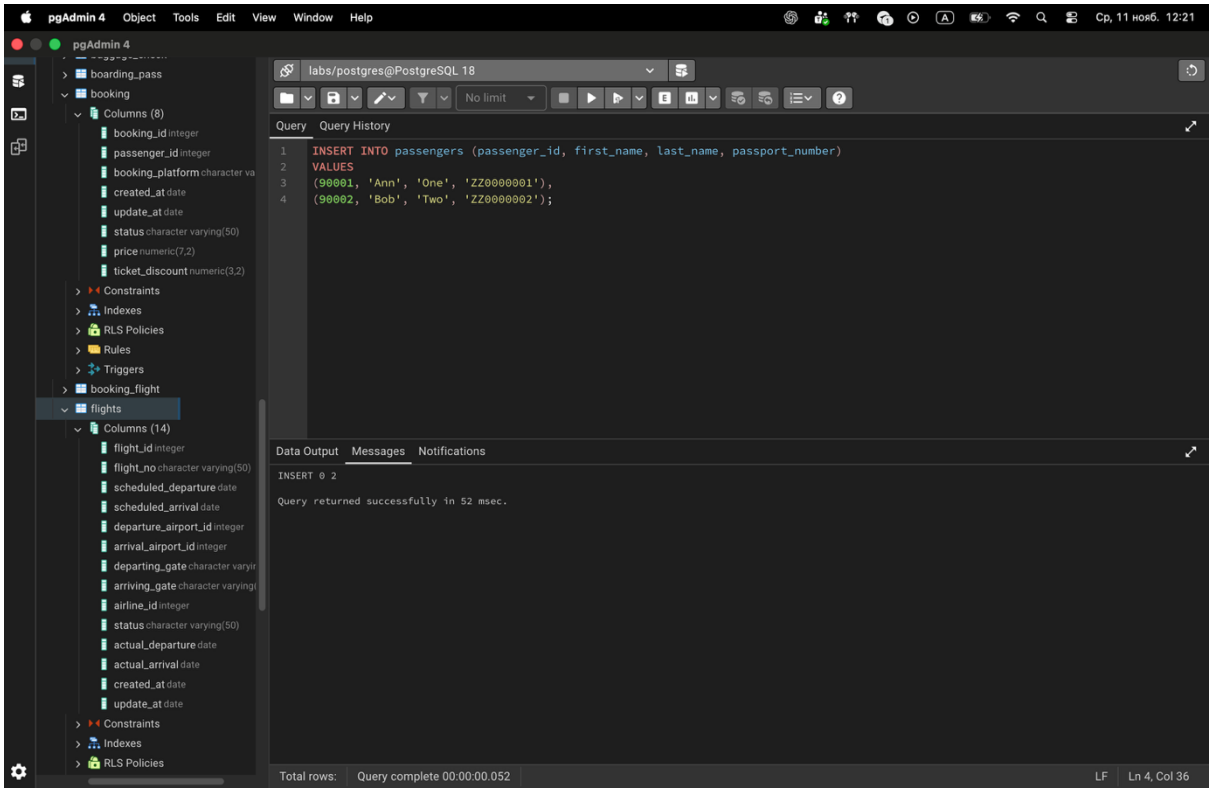
5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_airport.



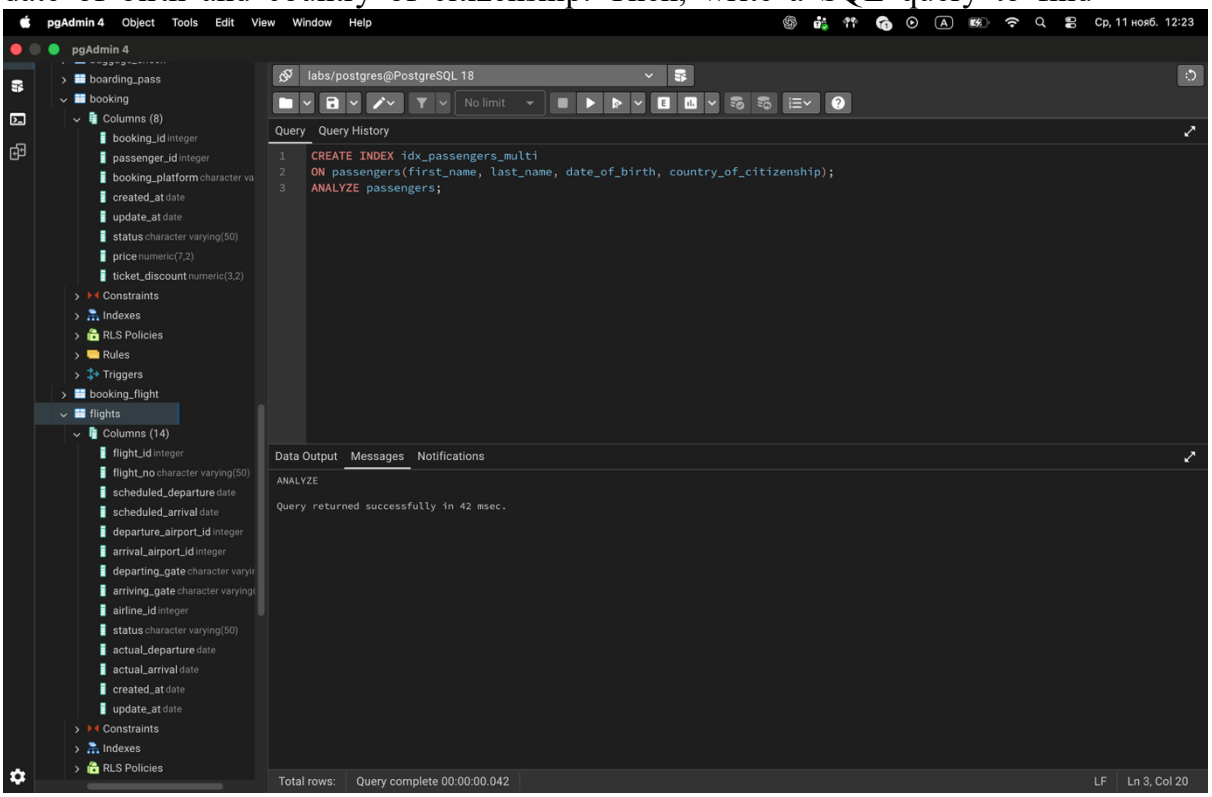
6. Create a unique index for the passport_number of the Passengers table. Check if the index was created or not. Insert into the table two new passengers. Explain in your own words what is going on in the output?


pgAdmin 4 Object Tools Edit View Window Help
Ср, 11 нояб. 12:20
pgAdmin 4
labs/postgres@PostgreSQL 18
No limit
Query Query History
1 CREATE UNIQUE INDEX uq_passengers_passport ON passengers(passport_number);
Data Output Messages Notifications
CREATE INDEX
Query returned successfully in 50 msec.
Total rows: Query complete 00:00:00.050 LF Ln 1, Col 75
boarding_pass
booking
Columns (8)
booking_id integer
passenger_id integer
booking_platform character va
created_at date
update_at date
status character varying(50)
price numeric(7,2)
ticket_discount numeric(3,2)
Constraints
Indexes
RLS Policies
Rules
Triggers
booking_flight
flights
Columns (14)
flight_id integer
flight_no character varying(50)
scheduled_departure date
scheduled_arrival date
departure_airport_id integer
arrival_airport_id integer
departing_gate character varyir
arriving_gate character varying(
airline_id integer
status character varying(50)
actual_departure date
actual_arrival date
created_at date
update_at date
Constraints
Indexes
RLS Policies

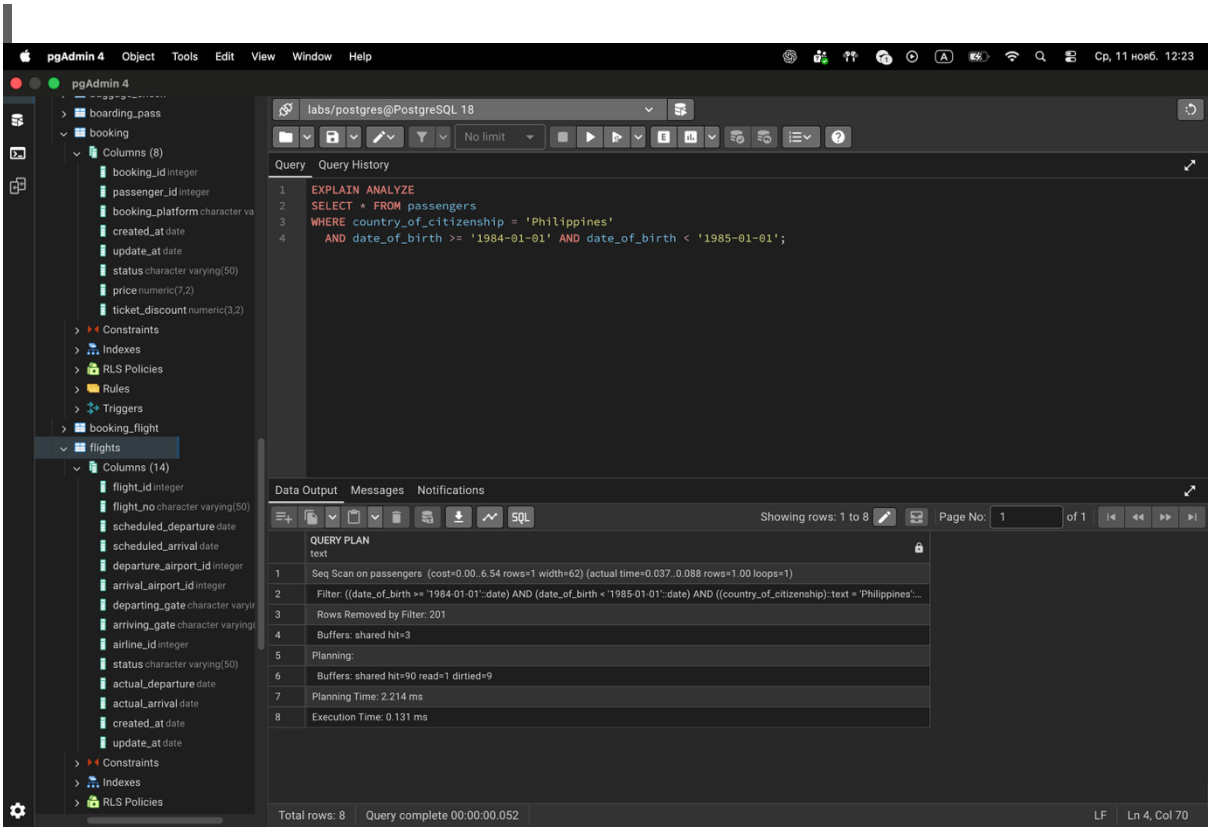

pgAdmin 4 Object Tools Edit View Window Help
Ср, 11 нояб. 12:20
pgAdmin 4
labs/postgres@PostgreSQL 18
No limit
Query Query History
1 SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'passengers';
Data Output Messages Notifications
Showing rows: 1 to 2 Page No: 1 of 1
indexname name
indexdef text
1 passengers_pkey CREATE UNIQUE INDEX passengers_pkey ON public.passengers USING btree (passenger_id)
2 uq_passengers_passp... CREATE UNIQUE INDEX uq_passengers_passport ON public.passengers USING btree (passport_nu...
Total rows: 2 Query complete 00:00:00.056 LF Ln 1, Col 75
boarding_pass
booking
Columns (8)
booking_id integer
passenger_id integer
booking_platform character va
created_at date
update_at date
status character varying(50)
price numeric(7,2)
ticket_discount numeric(3,2)
Constraints
Indexes
RLS Policies
Rules
Triggers
booking_flight
flights
Columns (14)
flight_id integer
flight_no character varying(50)
scheduled_departure date
scheduled_arrival date
departure_airport_id integer
arrival_airport_id integer
departing_gate character varyir
arriving_gate character varying(
airline_id integer
status character varying(50)
actual_departure date
actual_arrival date
created_at date
update_at date
Constraints
Indexes
RLS Policies



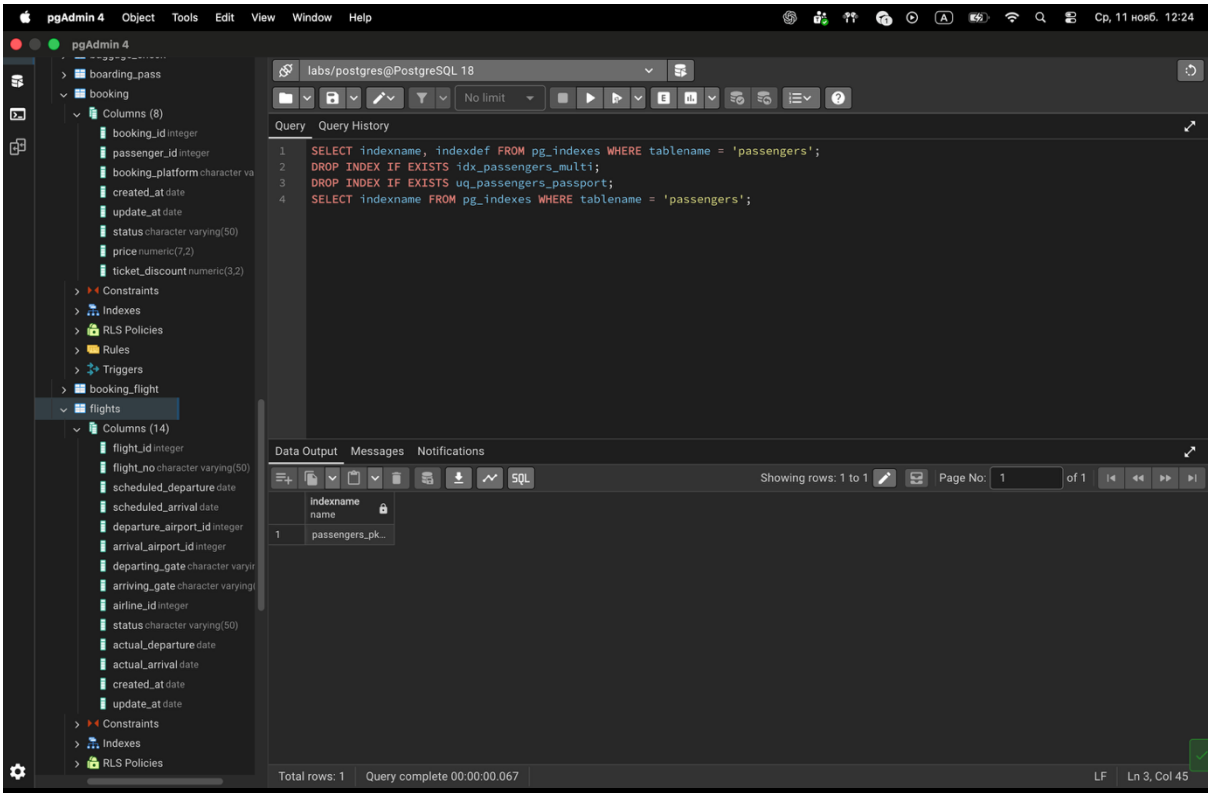
7. Create an index for the Passengers table. Use for that first name, last name, date of birth and country of citizenship. Then, write a SQL query to find

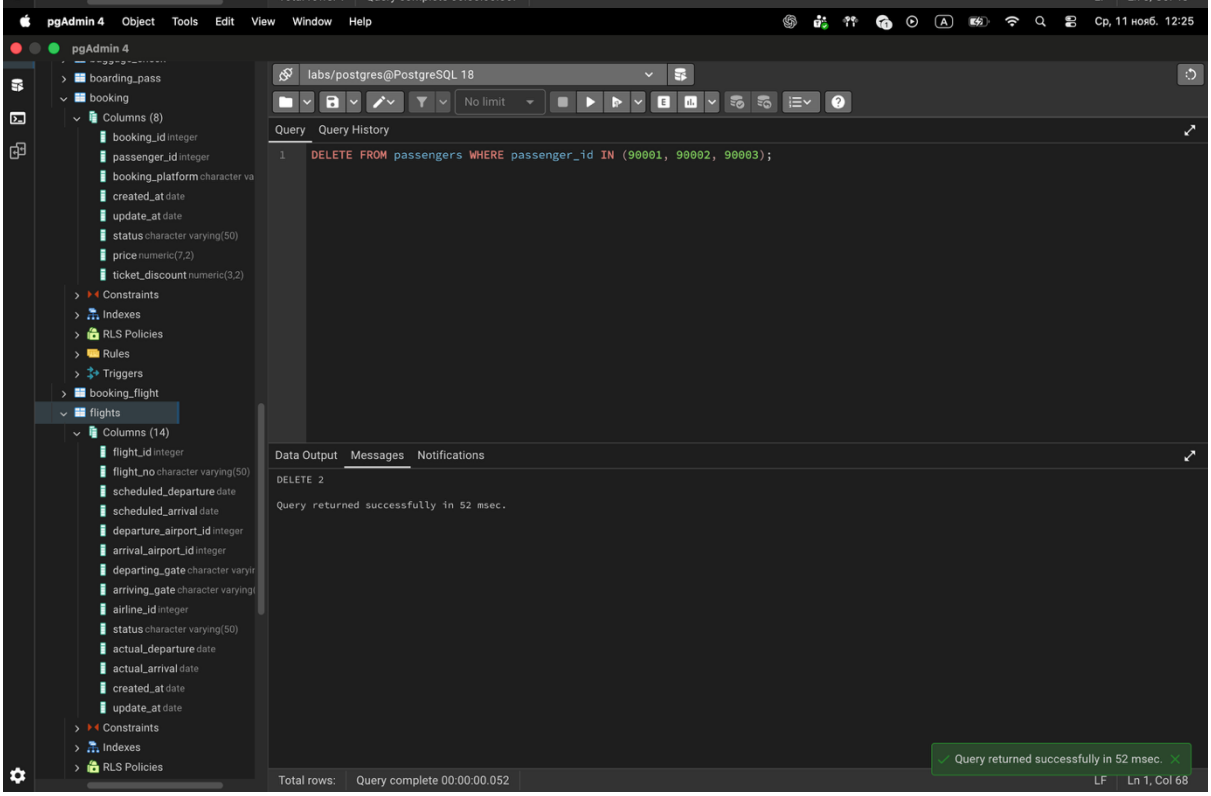


a passenger who was born in Philippines and was born in 1984 and check if the query uses indexes or not. Give the explanation of the results.

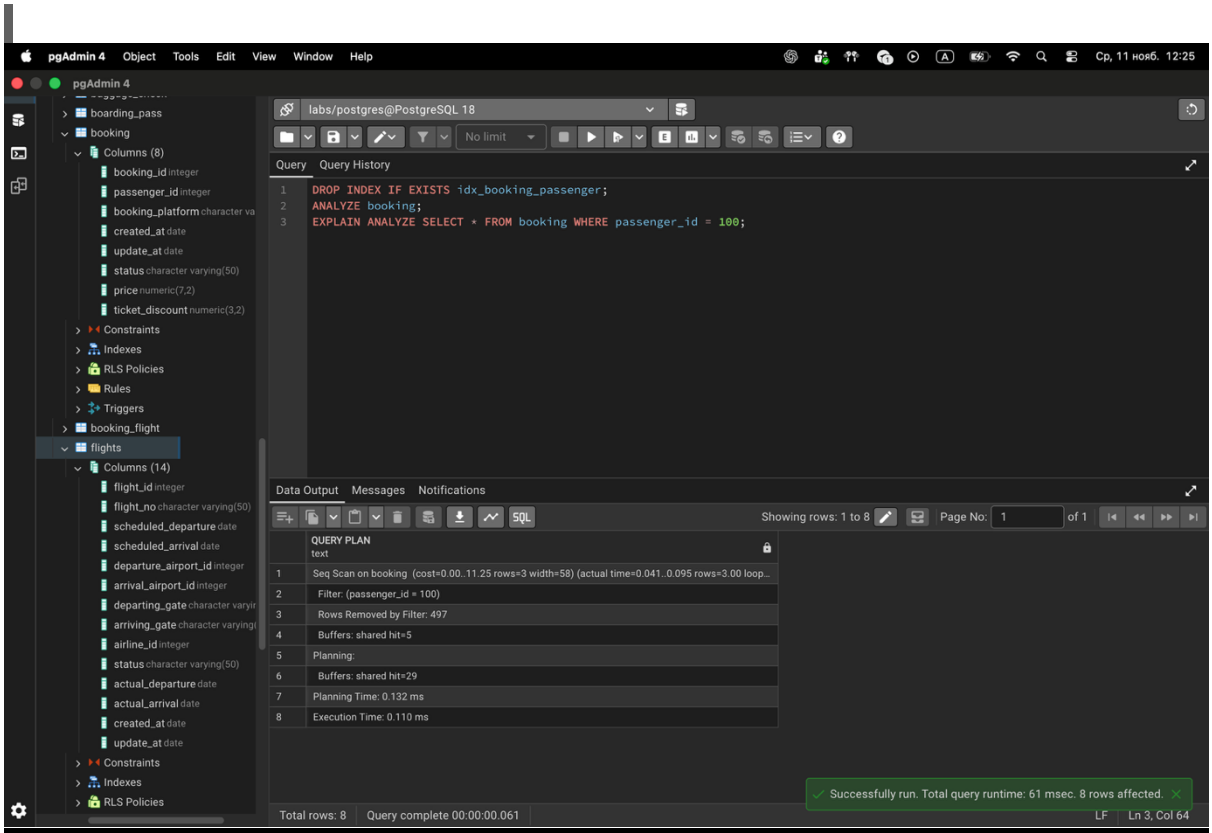


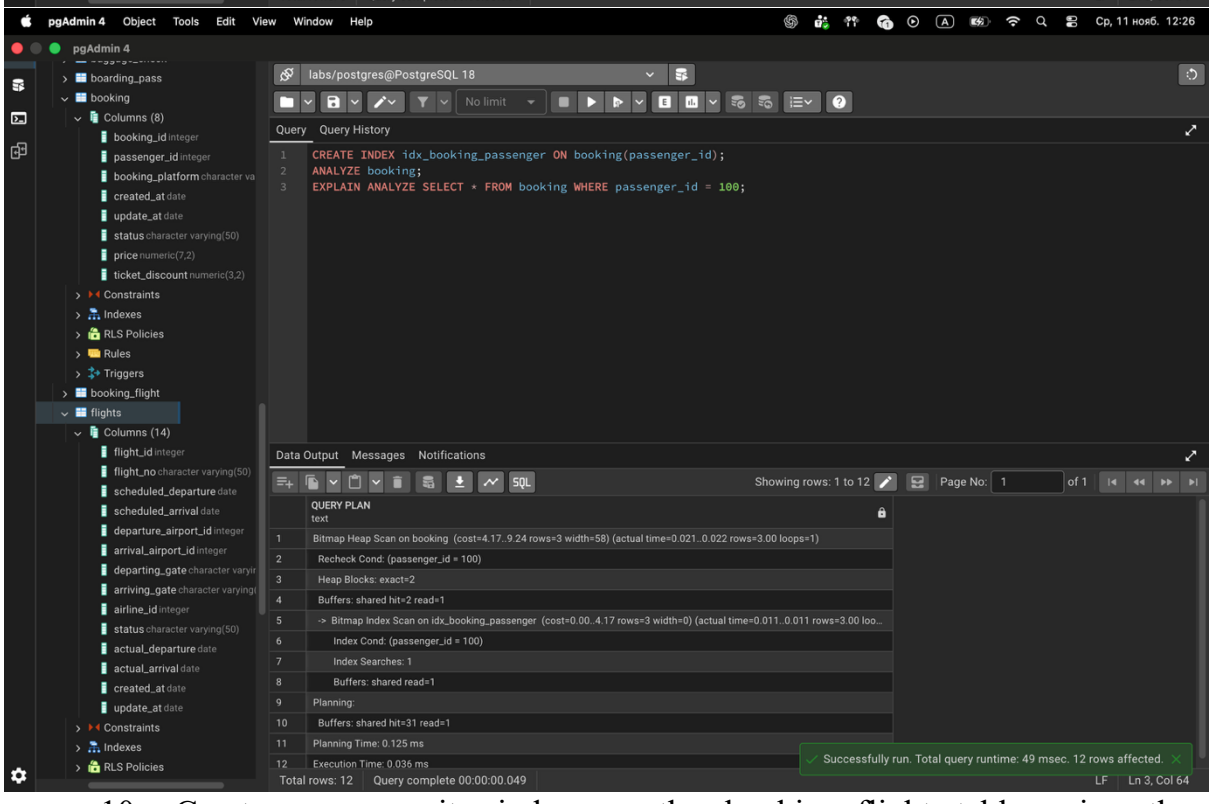
8. Write a SQL query to list indexes for table Passengers. After delete the created indexes.



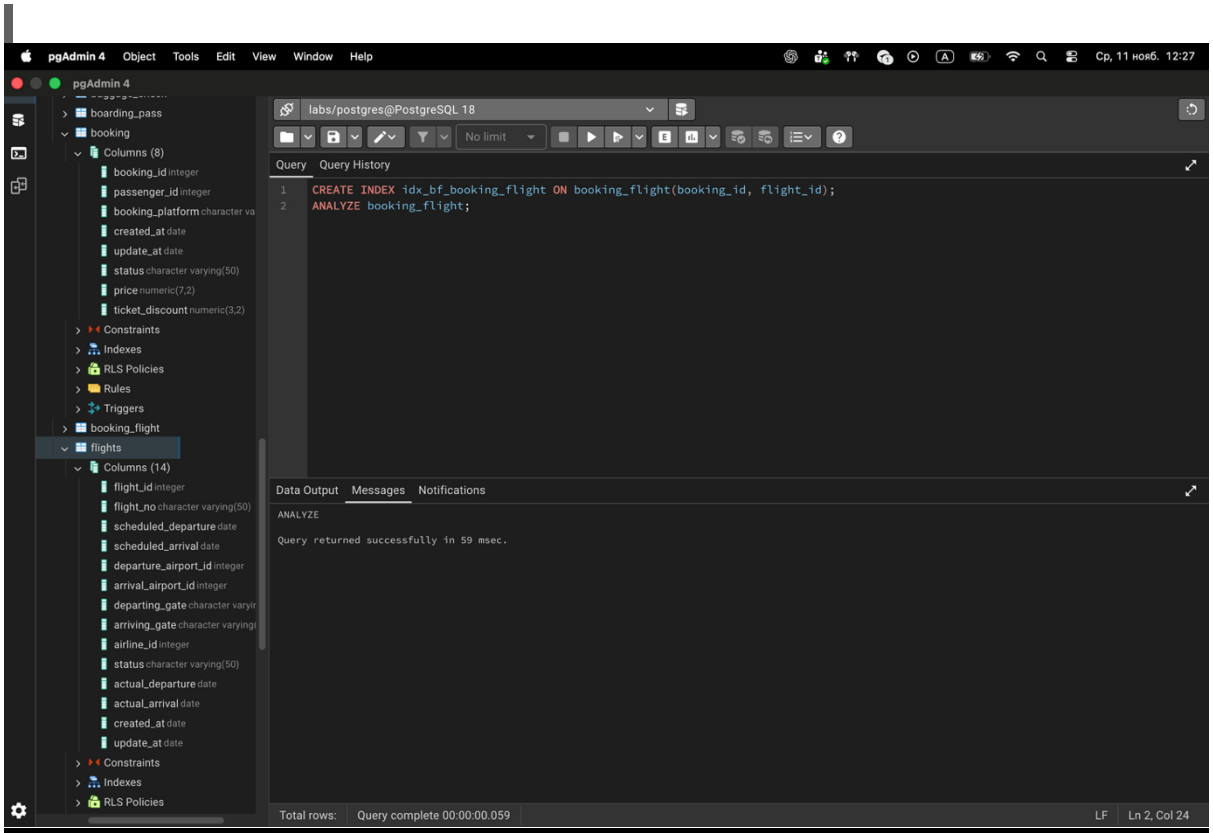


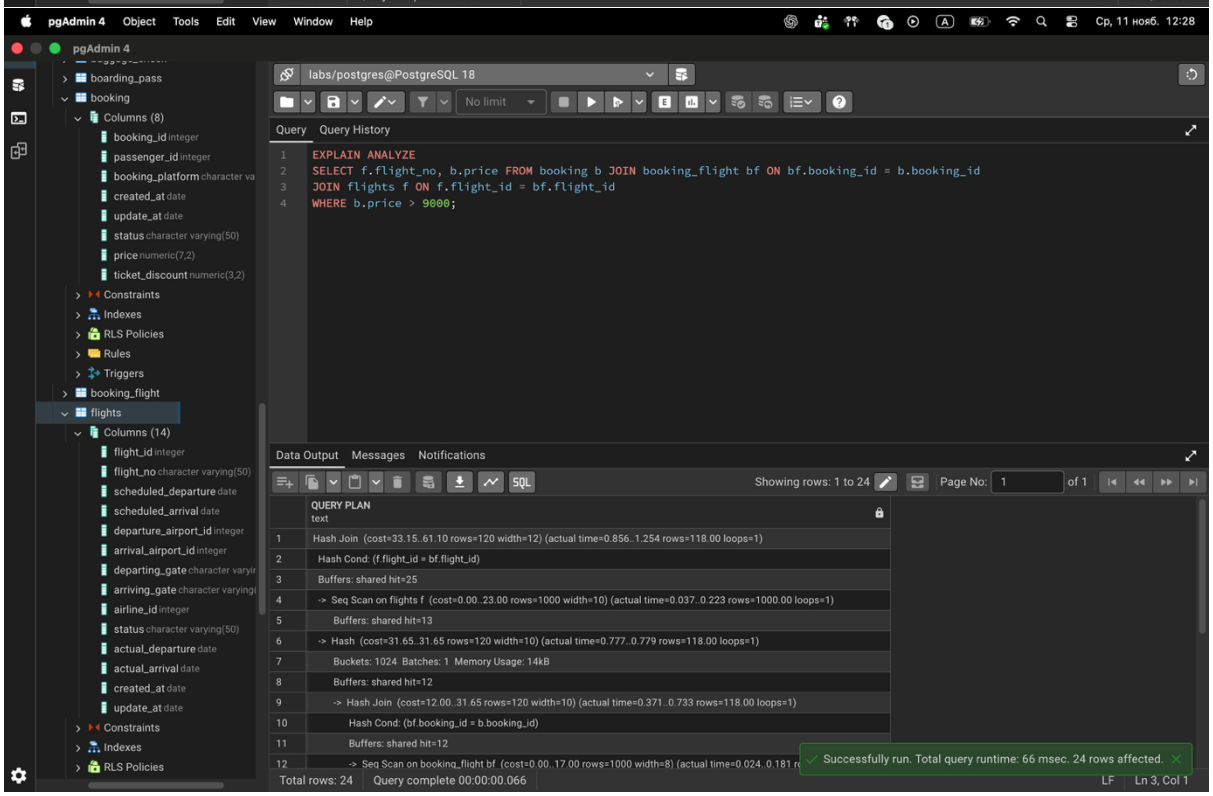
9. Create an index on the passenger_id column of the booking table. Using EXPLAIN ANALYZE, compare the execution plan and execution time of a query that selects all bookings of one passenger (for example, passenger_id = 100) before and after creating the index. Explain what changed in the plan.



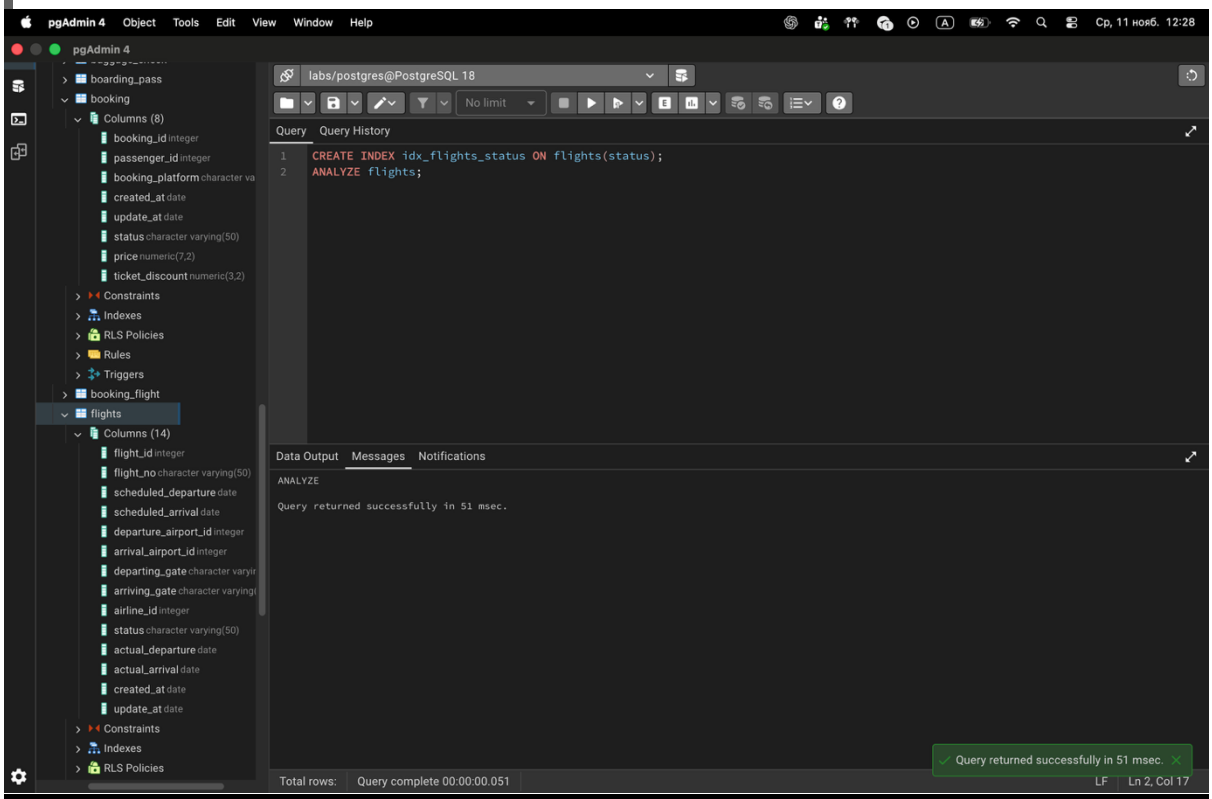


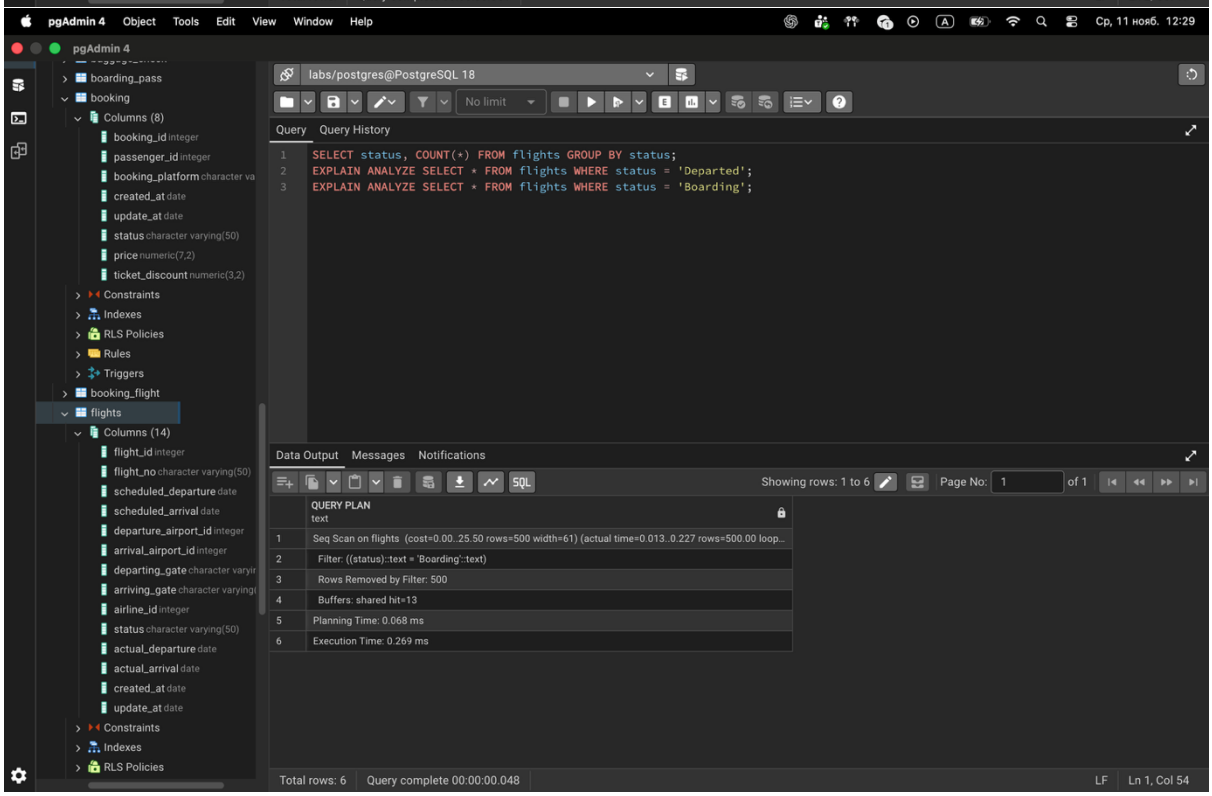
10. Create a composite index on the booking_flight table using the booking_id and flight_id columns. Write a query that joins booking, booking_flight and flights and returns the flight_no and price for bookings with a price greater than 9000. Check with EXPLAIN ANALYZE which indexes are used in each step of the join.



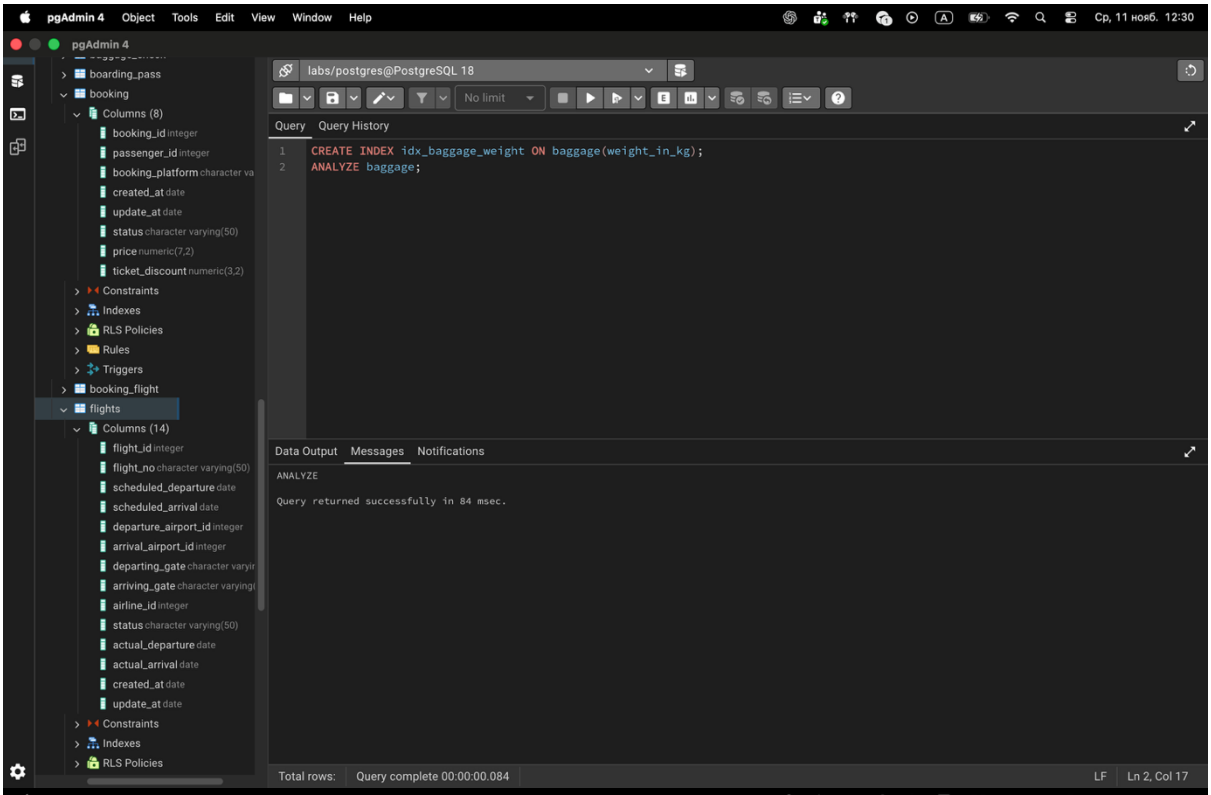


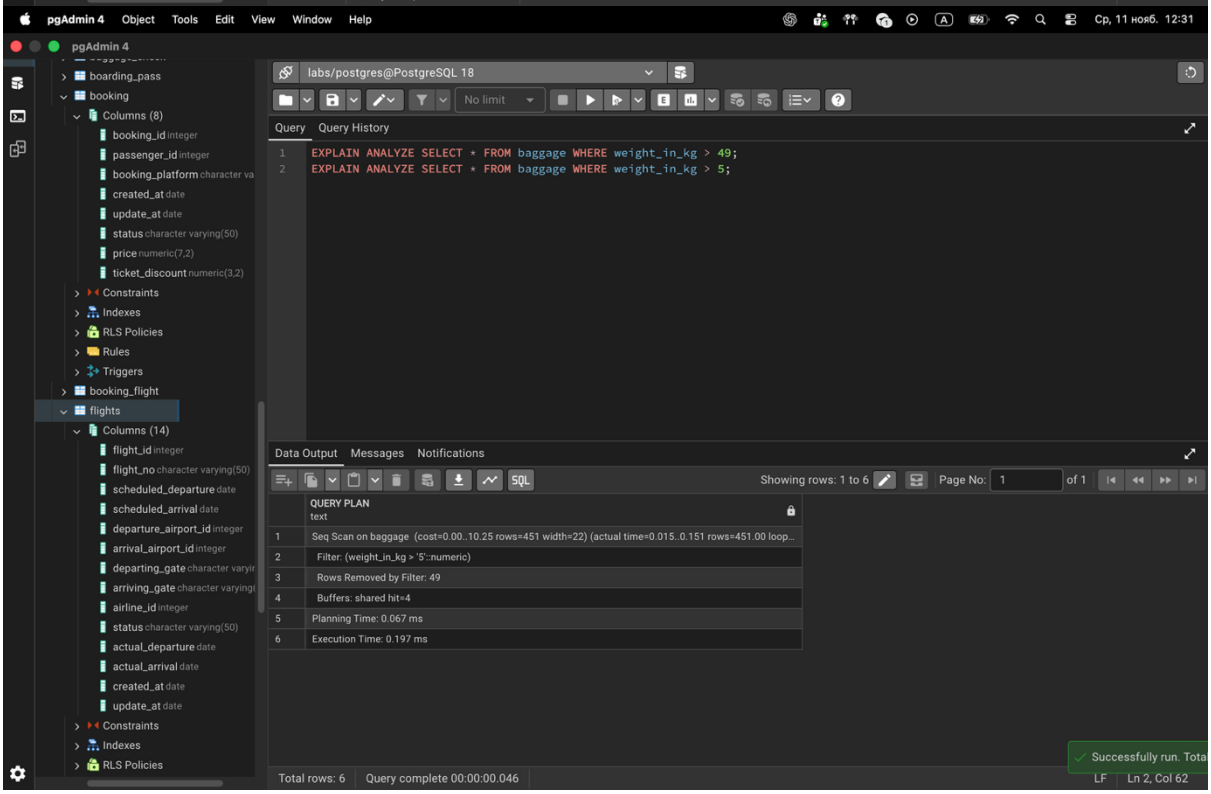
11. Create an index on the status column of the flights table. Run EXPLAIN ANALYZE for the query with status = 'Departed' and for the query with status = 'Boarding'. Compare the plans and explain, in your own words, why the optimizer may ignore the index for one of the values (hint: selectivity).





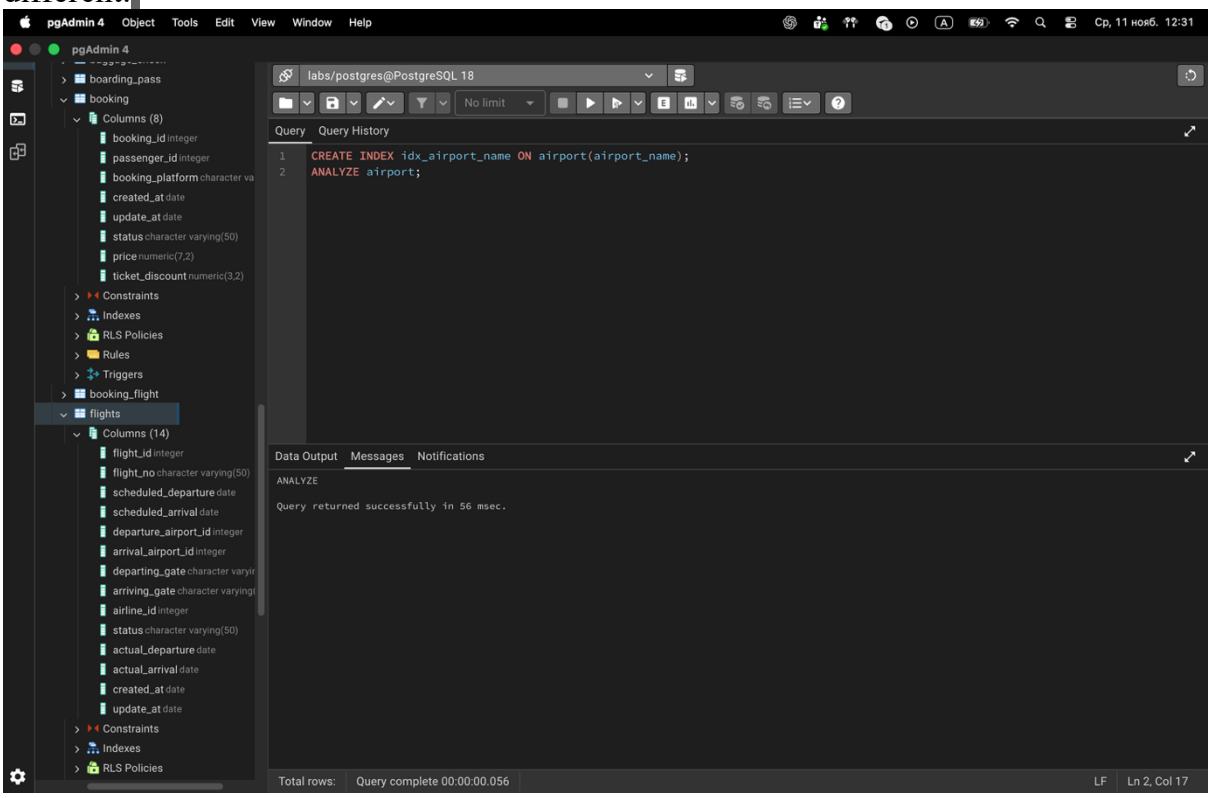
12. Create an index on the weight_in_kg column of the baggage table. Compare the plans of two range queries: weight_in_kg > 49 (a small part of the table) and weight_in_kg > 5 (almost the whole table). Explain the difference in index
usage.

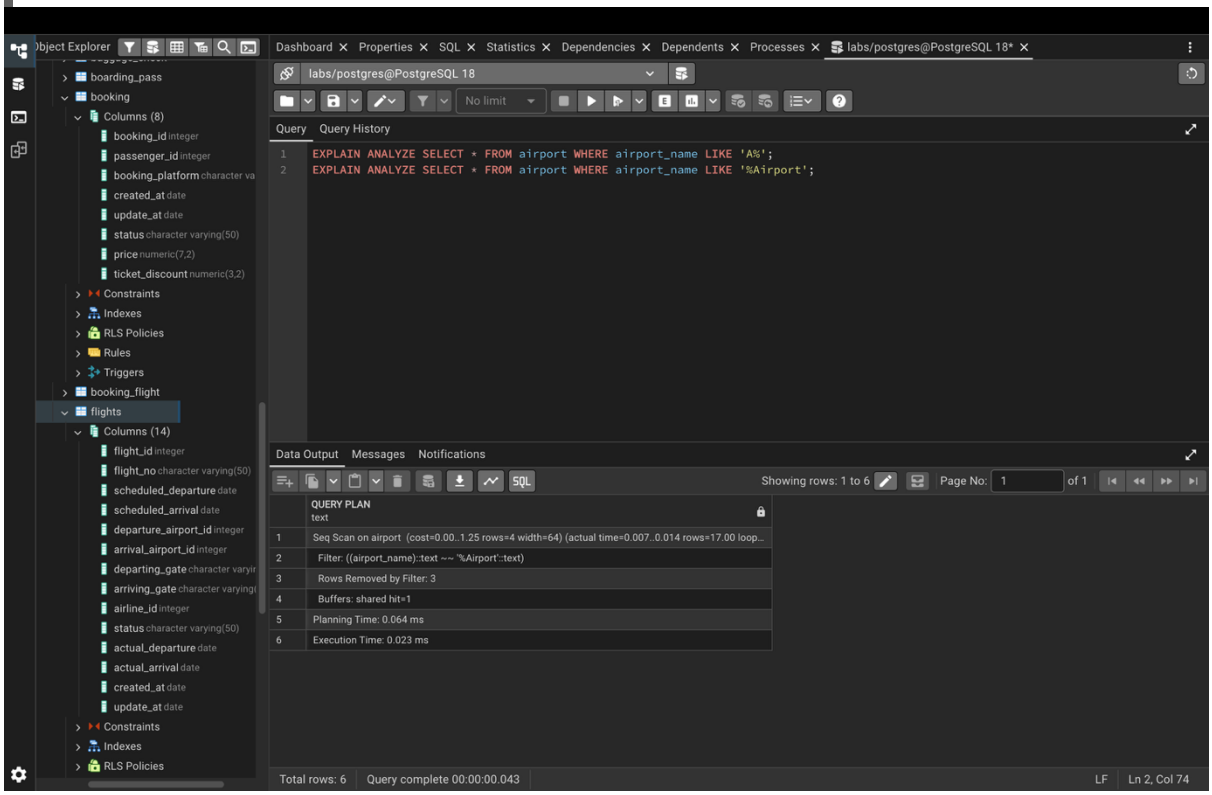




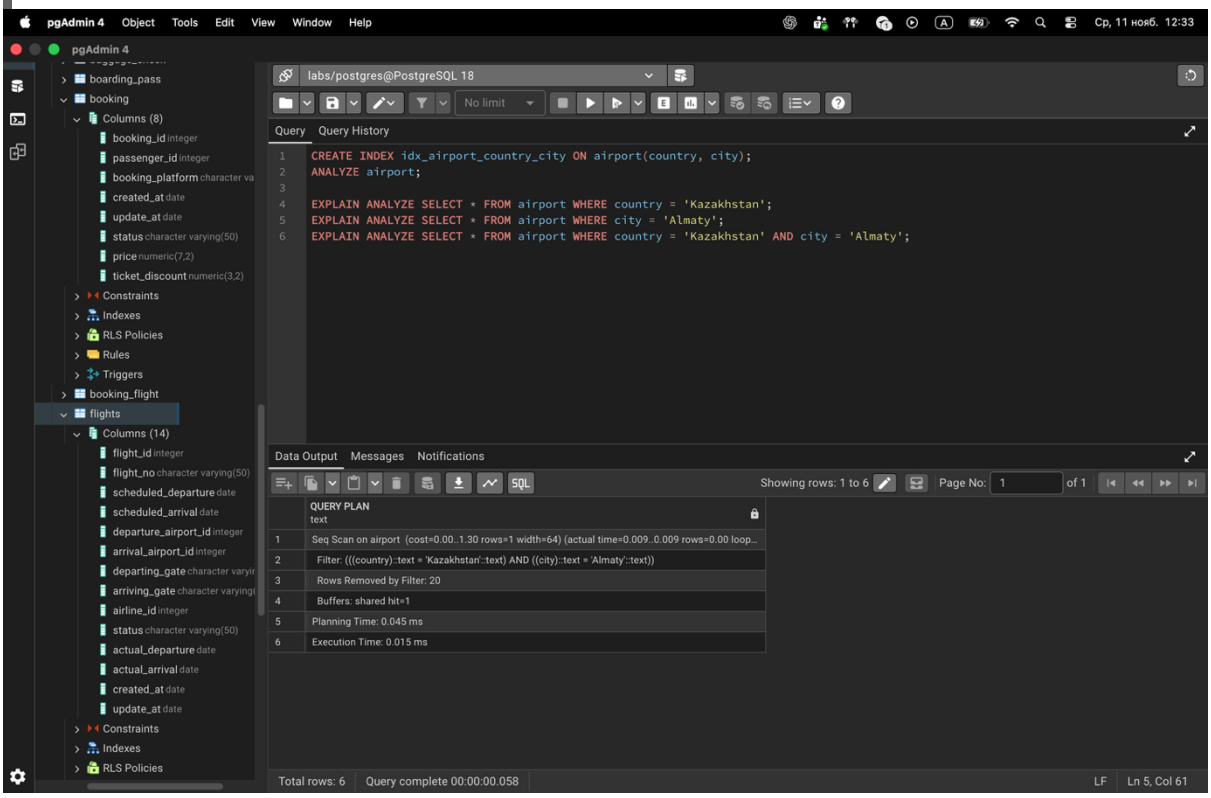
13. Create an index on the airport_name column of the airport table. Check with EXPLAIN ANALYZE whether it is used by the queries LIKE 'A%' and LIKE '%Airport'. Explain why the results are

different.

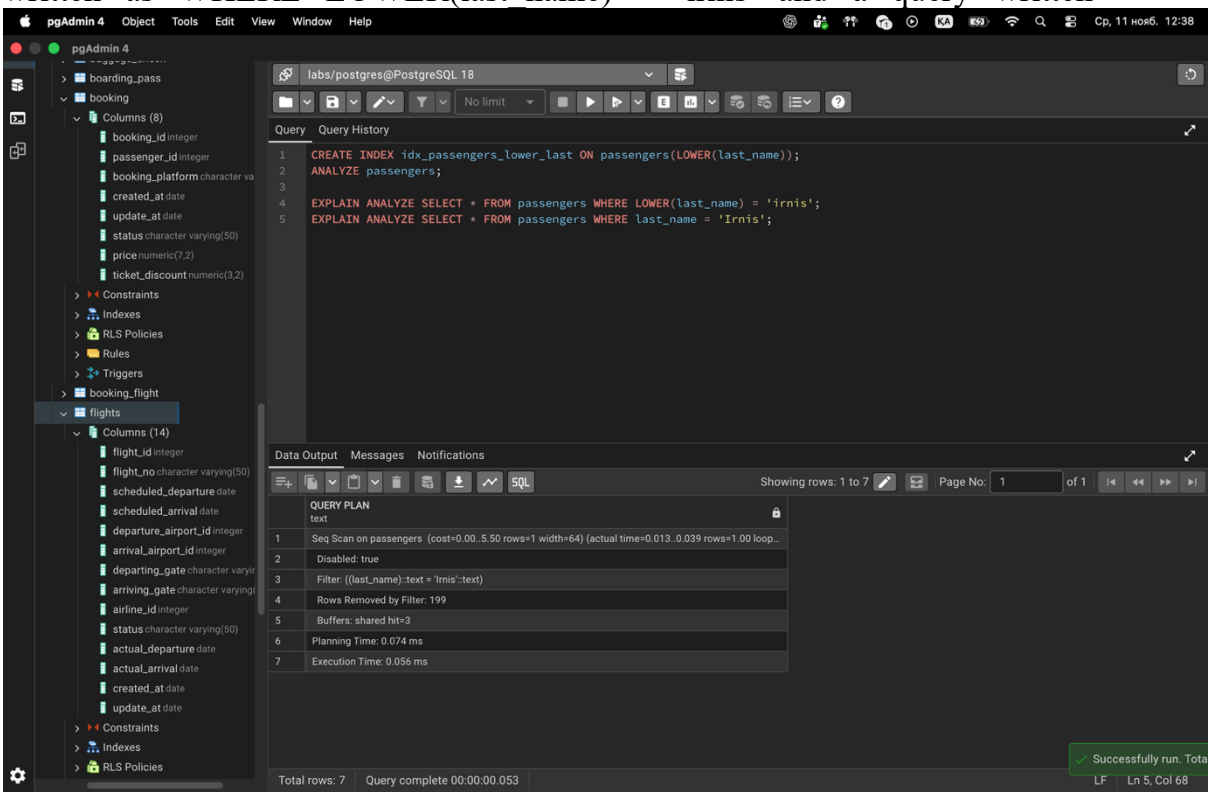




14. Create a composite index on the country and city columns of the airport table. Run three queries: filtering by country only, by city only, and by both columns. Check which of them use the index and explain the leftmost prefix rule using your results.

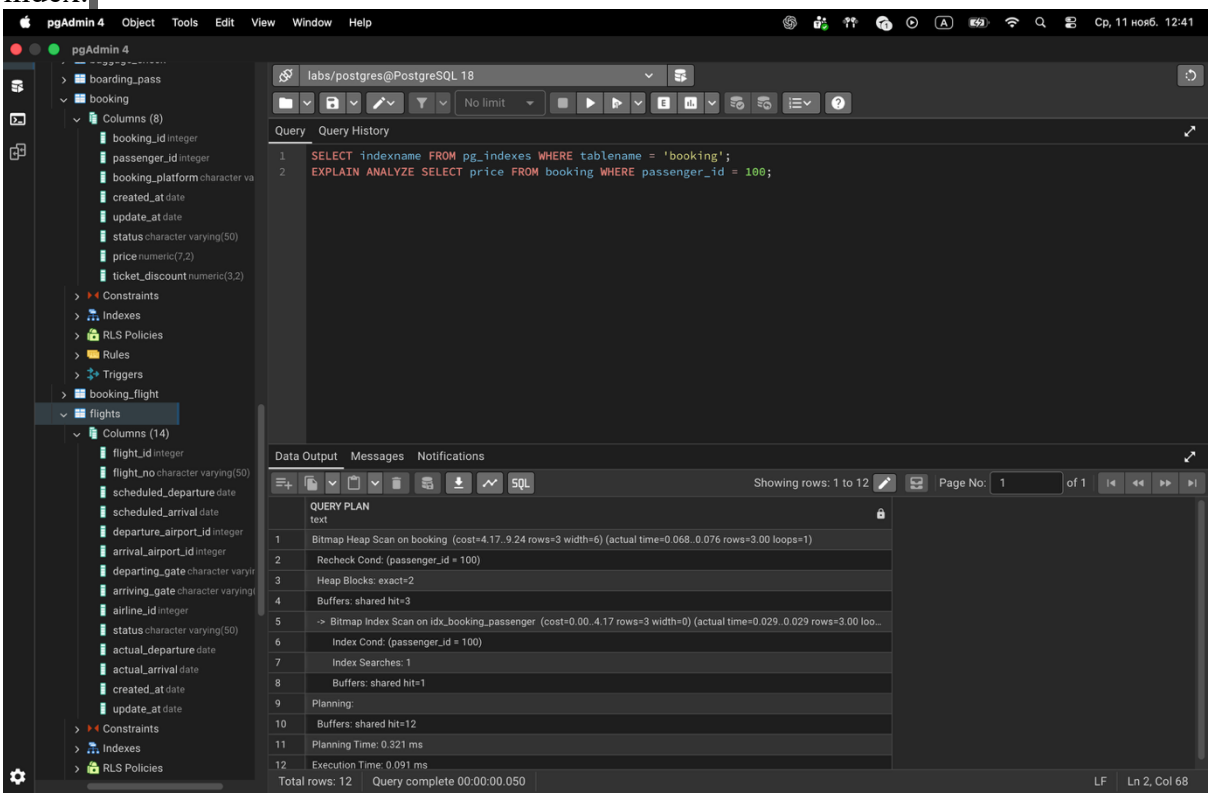


15. Create an expression (functional) index on the lower-case last_name of the Passengers table (LOWER(last_name)). Compare the plans of a query written as WHERE LOWER(last_name) = 'irnis' and a query written

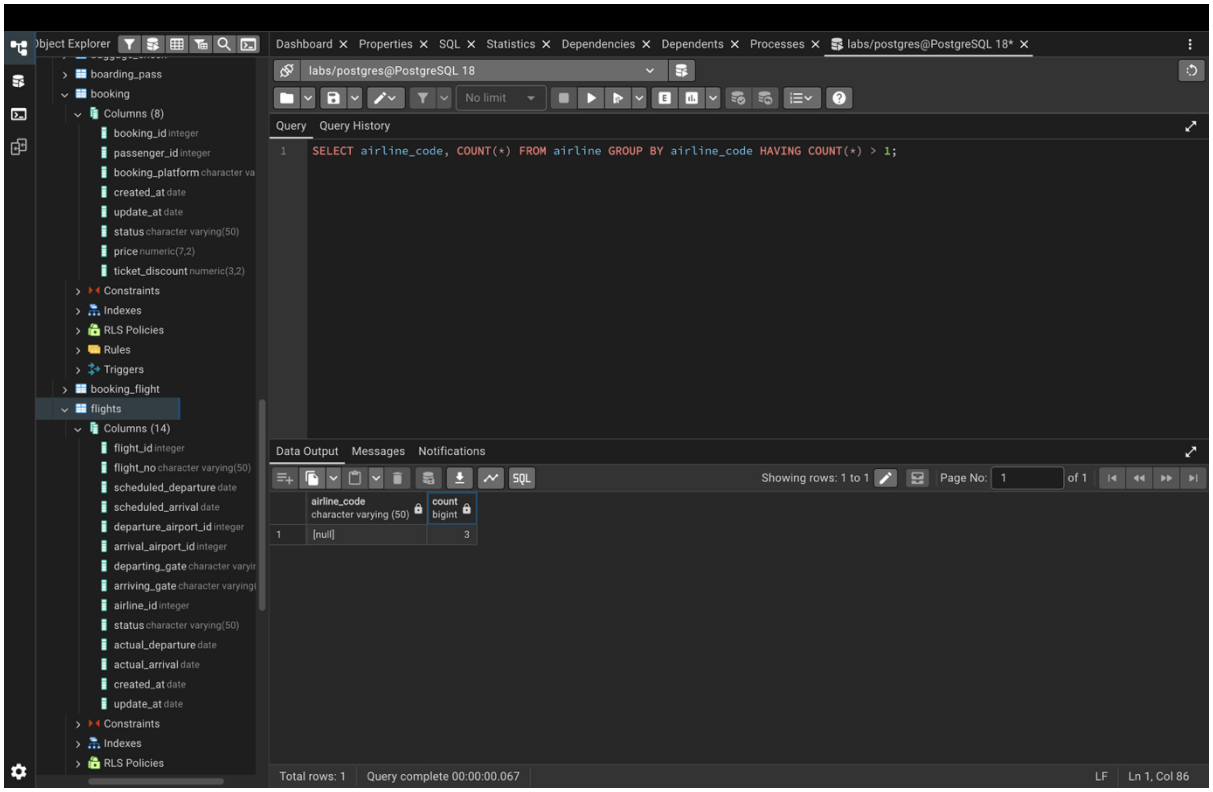


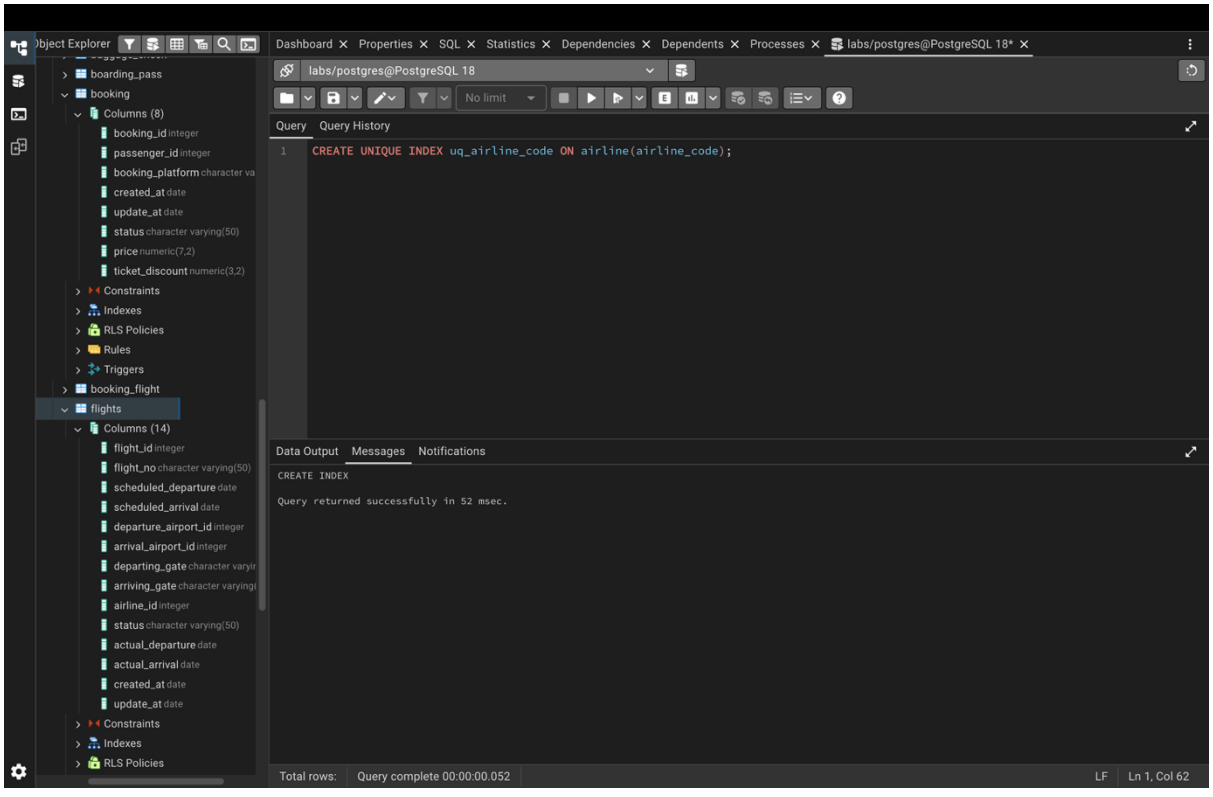
as WHERE last_name = 'Irnis'. Explain which query can use the index and why.

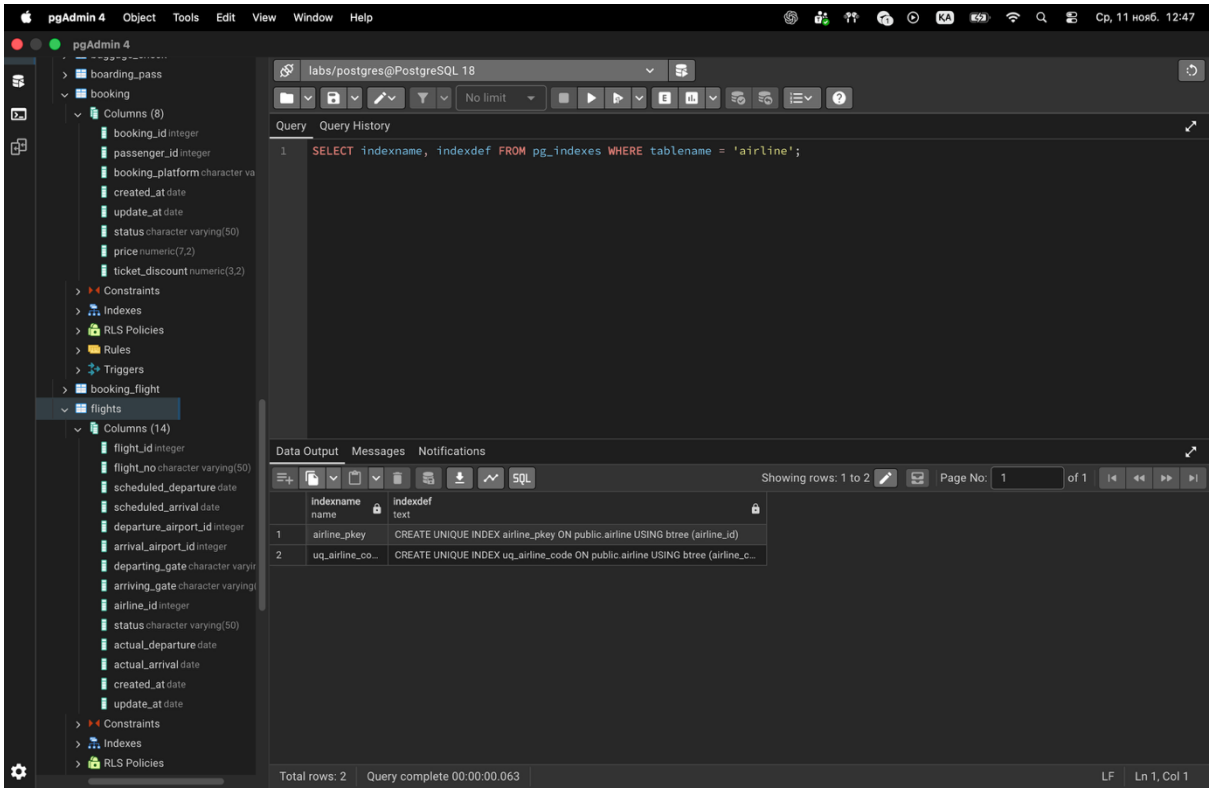
16. Create a covering index on the booking table for the columns passenger_id and price. Write a query that returns only price for a given passenger_id and check with EXPLAIN ANALYZE whether an index-only scan (or "Using index") is used. Explain the benefit of a covering index.



17. Try to create a unique index on the airline_code column of the airline table. If an error occurs, explain what caused it and find the problematic rows with a GROUP BY ... HAVING COUNT(*) > 1 query. Then create the unique index in a way that works for the current data, or after fixing the data.


Object Explorer
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
labs/postgres@PostgreSQL 18*
labs/postgres@PostgreSQL 18
No limit
Query
Query History
1 SELECT airline_code, COUNT(*) FROM airline GROUP BY airline_code HAVING COUNT(*) > 1;
Data Output
Messages
Notifications
Showing rows: 1 to 1
Page No: 1
of 1
airline_code character varying (50)
count bigint
1 [null] 3
Total rows: 1
Query complete 00:00:00.067
LF
Ln 1, Col 86


Object Explorer
Dashboard
Properties
SQL
Statistics
Dependencies
Dependents
Processes
labs/postgres@PostgreSQL 18*
labs/postgres@PostgreSQL 18
No limit
Query
Query History
1 CREATE UNIQUE INDEX uq_airline_code ON airline(airline_code);
Data Output
Messages
Notifications
CREATE INDEX
Query returned successfully in 52 msec.
Total rows:
Query complete 00:00:00.052
LF
Ln 1, Col 62


pgAdmin 4 Object Tools Edit View Window Help
Ср, 11 нояб. 12:47
pgAdmin 4
boarding_pass
booking
Columns (8)
booking_id integer
passenger_id integer
booking_platform character va
created_at date
update_at date
status character varying(50)
price numeric(7,2)
ticket_discount numeric(3,2)
Constraints
Indexes
RLS Policies
Rules
Triggers
booking_flight
flights
Columns (14)
flight_id integer
flight_no character varying(50)
scheduled_departure date
scheduled_arrival date
departure_airport_id integer
arrival_airport_id integer
departing_gate character varyir
arriving_gate character varying(
airline_id integer
status character varying(50)
actual_departure date
actual_arrival date
created_at date
update_at date
Constraints
Indexes
RLS Policies
labs/postgres@PostgreSQL 18
No limit
Query
Query History
1
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'airline';
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 2
Page No: 1
of 1
indexname name
indexdef text
1
airline_pkey
CREATE UNIQUE INDEX airline_pkey ON public.airline USING btree (airline_id)
2
uq_airline_co...
CREATE UNIQUE INDEX uq_airline_code ON public.airline USING btree (airline_c...
Total rows: 2
Query complete 00:00:00.063
LF
Ln 1, Col 1